

SRI KRISHNADEVARAYA EDUCATIONAL TRUST

SIR M. VISVESVARAYA INSTITUTE OF TECHNOLOGY

Krishnadevarayanagar, Hunasamaranahalli, International Air Port Road, Bangalore-562 157.

(Affiliated to Visvesvaraya Technological University, Recognised by AICTE & Accredited by National Board of Accreditation, New Delhi. An ISO 9001 : 2008 Certified Institution)

Ph. : 080-2846 7248, 2847 7024/25/26 Fax : 080-2846 7081

E-mail : principal@sirmvit.edu; sirmvitbgl@gmail.com, Web : www.sirmvit.edu



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING INTRODUCTION TO PYTHON PROGRAMMING LAB

BPLCKI05B/205B - 2ND SEMESTER B.E.

Academic Year – 2025-26

LAB MANUAL

Prepared by:
Dr. THRISHA V.S.
Assistant Professor
Dept. of CSE

Under the Guidance of:
Dr. T.N. ANITHA
Prof. and Head
Dept. of CSE

Department Vision and Mission

VISION

To create a center for imparting quality technical education and conducting research to meet the current and future challenges in the domain of computer science and engineering.

MISSION

- The Computer Science and Engineering department strives for excellence in teaching, applying, promoting and imparting knowledge through comprehensive academic programs.
- Train students to effectively apply the knowledge to solve real-world problems, thus enhance their potential for life-long high-quality career and give them a competitive advantage in the ever-changing and fast paced computing world.
- Prepare students to demonstrate a sense of societal and ethical responsibilities in their professional endeavors.
- Creating amongst students and faculty a collaborative environment open to the free exchange of ideas, where research, creativity, innovation and entrepreneurship can flourish.

PRACTICAL COMPONENTS OF IPCC

PART – A: FIXED SET OF EXPERIMENTS

1.
 - a. Develop a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide).
 - b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.
 2.
 - a. Develop a program to generate Fibonacci sequence of length (N). Read N from the console.
 - b. Write a python program to create a list and perform the following operations
 - Inserting an element
 - Removing an element
 - Appending an element
 - Displaying the length of the list
 - Popping an element
 - Clearing the list
 3.
 - a. Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.
 - b. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.
 4. Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use a dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display the dictionary slice of the first 10 items.
 5. Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimensional array and sort using the Bubble Sort technique].
 6. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write()].
-
7. Develop a function named DivExp which takes TWO parameters a, b, and returns a value c ($c=a/b$). Write a suitable assertion for $a>0$ in the function DivExp and raise an exception for when $b=0$. Develop a suitable program that reads two console values and calls the function DivExp.
 8. Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class 'Complex' to represent the complex number. Develop a program to read N ($N \geq 2$) complex numbers and to compute the addition of N complex numbers.
 9. Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.
 10. Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.
 11. Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).
 12. Develop a program to display contents of a folder recursively (Directory) having sub-folders and files (name and type).

BPLCK205 – INTRODUCTION TO PYTHON PROGRAMMING

1a) Develop a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide).

```
# Program to perform basic arithmetic operations

# Taking input from the user
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("\nSelect operation:")
print("1 - Add")
print("2 - Subtract")
print("3 - Multiply")
print("4 - Divide")

choice = input("Enter your choice (1/2/3/4): ")

# Performing operation based on choice
if choice == '1':
    result = num1 + num2
    print("Result =", result)

elif choice == '2':
    result = num1 - num2
    print("Result =", result)

elif choice == '3':
    result = num1 * num2
    print("Result =", result)

elif choice == '4':
    if num2 != 0:
        result = num1 / num2
        print("Result =", result)
    else:
        print("Error! Division by zero is not allowed.")

else:
    print("Invalid choice! Please select a valid option (1-4).")
```

Output:

Enter first number: 10

Enter second number: 5

Select operation:

1 - Add

2 - Subtract

3 - Multiply

4 - Divide

Enter your choice (1/2/3/4): 3

Result = 50.0

1b) Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.

```
# Program to check if a person is a senior citizen
```

```
from datetime import datetime
```

```
# Taking input
```

```
name = input("Enter your name: ")
```

```
year_of_birth = int(input("Enter your year of birth: "))
```

```
# Getting current year
```

```
current_year = datetime.now().year
```

```
# Calculating age
```

```
age = current_year - year_of_birth
```

```
# Checking senior citizen status
```

```
print("\nName:", name)
```

```
print("Age:", age)
```

```
if age >= 60:
```

```
    print("Status: Senior Citizen")
```

```
else:
```

```
    print("Status: Not a Senior Citizen")
```

Output:

Enter your name: Ramesh

Enter your year of birth: 1955

Name: Ramesh

Age: 71

Status: Senior Citizen

2a) Develop a program to generate Fibonacci sequence of length (N). Read N from the console

```
# Program to generate Fibonacci sequence of length N

# Read N from the user
N = int(input("Enter the value of N: "))

# Check if N is valid
if N <= 0:
    print("Please enter a positive integer.")
else:
    fib_sequence = []

    # First two Fibonacci numbers
    a, b = 0, 1

    for i in range(N):
        fib_sequence.append(a)
        a, b = b, a + b

    print("Fibonacci sequence of length", N, "is:")
    print(fib_sequence)
```

Output:

```
Enter the value of N: 7
Fibonacci sequence of length 7 is:
[0, 1, 1, 2, 3, 5, 8]
```

2b) Write a python program to create a list and perform the following operations

- **Inserting an element**
- **Removing an element**
- **Appending an element**
- **Displaying the length of the list**
- **Popping an element**
- **Clearing the list**

```
# Program to demonstrate list operations

# Creating an empty list
my_list = []
```

```

while True:
    print("\n--- List Operations Menu ---")
    print("1. Insert an element")
    print("2. Remove an element")
    print("3. Append an element")
    print("4. Display length of the list")
    print("5. Pop an element")
    print("6. Clear the list")
    print("7. Display the list")
    print("8. Exit")

    choice = input("Enter your choice (1-8): ")

    if choice == '1':
        element = input("Enter element to insert: ")
        position = int(input("Enter position to insert at: "))
        my_list.insert(position, element)
        print("Element inserted successfully.")

    elif choice == '2':
        element = input("Enter element to remove: ")
        if element in my_list:
            my_list.remove(element)
            print("Element removed successfully.")
        else:
            print("Element not found in the list.")

    elif choice == '3':
        element = input("Enter element to append: ")
        my_list.append(element)
        print("Element appended successfully.")

    elif choice == '4':
        print("Length of the list:", len(my_list))

    elif choice == '5':
        if my_list:
            popped_element = my_list.pop()
            print("Popped element:", popped_element)
        else:
            print("List is empty. Nothing to pop.")

    elif choice == '6':
        my_list.clear()
        print("List cleared successfully.")

```

```
elif choice == '7':
    print("Current List:", my_list)

elif choice == '8':
    print("Exiting program.")
    break

else:
    print("Invalid choice! Please select a valid option.")
```

Output:

```
--- List Operations Menu ---
1. Insert an element
2. Remove an element
3. Append an element
4. Display length of the list
5. Pop an element
6. Clear the list
7. Display the list
8. Exit
Enter your choice (1-8): 3
Enter element to append: 10
Element appended successfully.
```

```
--- List Operations Menu ---
Enter your choice (1-8): 3
Enter element to append: 20
Element appended successfully.
```

```
--- List Operations Menu ---
Enter your choice (1-8): 7
Current List: ['10', '20']
```

```
--- List Operations Menu ---
Enter your choice (1-8): 1
Enter element to insert: 15
Enter position to insert at: 1
Element inserted successfully.
```

```
--- List Operations Menu ---
Enter your choice (1-8): 7
Current List: ['10', '15', '20']
```

```
--- List Operations Menu ---
Enter your choice (1-8): 2
```

Enter element to remove: 15
Element removed successfully.

--- List Operations Menu ---
Enter your choice (1-8): 7
Current List: ['10', '20']

--- List Operations Menu ---
Enter your choice (1-8): 4
Length of the list: 2

--- List Operations Menu ---
Enter your choice (1-8): 5
Popped element: 20

--- List Operations Menu ---
Enter your choice (1-8): 7
Current List: ['10']

--- List Operations Menu ---
Enter your choice (1-8): 6
List cleared successfully.

--- List Operations Menu ---
Enter your choice (1-8): 7
Current List: []

--- List Operations Menu ---
Enter your choice (1-8): 8
Exiting program.

3a) Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.

```
# Program to calculate Mean, Variance and Standard Deviation
```

```
import math
```

```
# Read value of N
```

```
N = int(input("Enter the number of elements: "))
```

```
numbers = []
```

```
# Read N numbers
```

```
for i in range(N):
```

```

num = float(input(f"Enter number {i+1}: "))
numbers.append(num)

# Calculate Mean
mean = sum(numbers) / N

# Calculate Variance
variance = sum((x - mean) ** 2 for x in numbers) / N

# Calculate Standard Deviation
std_deviation = math.sqrt(variance)

# Display results
print("\nNumbers:", numbers)
print("Mean =", mean)
print("Variance =", variance)
print("Standard Deviation =", std_deviation)

```

Output:

```

Enter the number of elements: 5
Enter number 1: 10
Enter number 2: 20
Enter number 3: 30
Enter number 4: 40
Enter number 5: 50

Numbers: [10.0, 20.0, 30.0, 40.0, 50.0]
Mean = 30.0
Variance = 200.0
Standard Deviation = 14.142135623730951

```

3b) Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.

```

# Program to count frequency of each digit in a number

# Read number as string (characters)
number = input("Enter a multi-digit number: ")

# Dictionary to store digit frequency
digit_count = {}

# Count frequency of each digit
for digit in number:
    if digit.isdigit(): # Ensure it is a digit
        if digit in digit_count:

```

```

        digit_count[digit] += 1
    else:
        digit_count[digit] = 1

# Display results
print("\nDigit Frequency:")

for digit in sorted(digit_count):
    print(f"Digit {digit} occurs {digit_count[digit]} time(s)")

```

Output:

Enter a multi-digit number: 122345522

Digit Frequency:
 Digit 1 occurs 1 time(s)
 Digit 2 occurs 4 time(s)
 Digit 3 occurs 1 time(s)
 Digit 4 occurs 1 time(s)
 Digit 5 occurs 2 time(s)

4)Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use a dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display the dictionary slice of the first 10 items.

```
# Program to print 10 most frequently appearing words in a text file
```

```
# Open the file (change filename as needed)
filename = input("Enter the file name: ")
```

```
with open(filename, 'r') as file:
    text = file.read()
```

```
# Convert text to lowercase and split into words
words = text.lower().split()
```

```
# Remove punctuation from words
cleaned_words = []
for word in words:
    word = word.strip(",.!?;:\\""()[]{}")
    if word != "":
        cleaned_words.append(word)
```

```
# Dictionary to store word frequencies
word_count = {}
```

```
for word in cleaned_words:
```

```

if word in word_count:
    word_count[word] += 1
else:
    word_count[word] = 1

# Sort dictionary by frequency (descending order)
sorted_words = sorted(word_count.items(), key=lambda x: x[1], reverse=True)

# Display top 10 words
print("\nTop 10 most frequently appearing words:\n")

for word, count in sorted_words[:10]:
    print(f"{word} : {count}")

```

Output:

Enter the file name: sample.txt

Top 10 most frequently appearing words:

```

the : 25
and : 18
to : 15
of : 14
a : 12
in : 10
is : 9
that : 8
it : 7
for : 6

```

5) Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimesional array and sort using the Bubble Sort technique].

```

# Program to sort 6 subject marks in descending order using Bubble Sort

```

```

# Read 6 subject marks
marks = []

```

```

print("Enter marks for 6 subjects:")

```

```

for i in range(6):
    mark = float(input(f"Enter mark {i+1}: "))
    marks.append(mark)

```

```

# Bubble Sort (Descending Order)

```

```

n = len(marks)

for i in range(n):
    for j in range(0, n - i - 1):
        if marks[j] < marks[j + 1]: # Swap for descending order
            marks[j], marks[j + 1] = marks[j + 1], marks[j]

# Display sorted marks
print("\nMarks in descending order (Highest to Lowest):")
for mark in marks:
    print(mark)

```

Output:

```

Enter marks for 6 subjects:
Enter mark 1: 78
Enter mark 2: 85
Enter mark 3: 67
Enter mark 4: 90
Enter mark 5: 88
Enter mark 6: 72

```

```

Marks in descending order (Highest to Lowest):
90.0
88.0
85.0
78.0
72.0
67.0

```

6) Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write().]

```

# Program to sort contents of a text file
# and write sorted contents into another file

# Input and output file names
input_file = input("Enter input file name: ")
output_file = input("Enter output file name: ")

# Open and read lines from input file
with open(input_file, 'r') as file:
    lines = file.readlines()

# Create an empty list to store cleaned lines
cleaned_lines = []

```

```
# Remove newline characters and empty lines
for line in lines:
    line = line.strip()    # remove leading/trailing spaces & newline
    if len(line) > 0:      # ignore empty lines
        cleaned_lines.append(line)

# Sort the list
cleaned_lines.sort()

# Write sorted lines into output file
with open(output_file, 'w') as file:
    for line in cleaned_lines:
        file.write(line + "\n")

print("\nFile sorted successfully.")
print("Sorted content written to", output_file)
```

Output:

Input file (input.txt):

Code

```
Banana
Apple
Mango
Orange
Grapes
```

Output file (output.txt):

Code

```
Apple
Banana
Grapes
Mango
Orange
```

7) Develop a function named DivExp which takes TWO parameters a, b, and returns a value c ($c=a/b$). Write a suitable assertion for $a>0$ in the function DivExp and raise an exception for when $b=0$. Develop a suitable program that reads two console values and calls the function DivExp.

```
# Function DivExp: divides a by b
def DivExp(a, b):
```

```

# Assertion: a must be greater than 0
assert a > 0, "Error: 'a' must be greater than 0"

# Raise exception if b is zero
if b == 0:
    raise ValueError("Error: Division by zero is not allowed")

c = a / b
return c

# Main program
try:
    a = float(input("Enter value for a: "))
    b = float(input("Enter value for b: "))

    result = DivExp(a, b)
    print(f"Result of {a} / {b} = {result}")

except AssertionError as ae:
    print(ae)

except ValueError as ve:
    print(ve)

except Exception as e:
    print("An unexpected error occurred:", e)

```

Output:

Example 1: Valid input

Code

```

Enter value for a: 10
Enter value for b: 2
Result of 10.0 / 2.0 = 5.0

```

Example 2: $a \leq 0$ (Assertion triggers)

Code

```

Enter value for a: -5
Enter value for b: 3
Error: 'a' must be greater than 0

```

Example 3: $b = 0$ (Exception triggers)

Code

```

Enter value for a: 10
Enter value for b: 0
Error: Division by zero is not allowed

```

8) Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class 'Complex' to represent the complex number. Develop a program to read N ($N \geq 2$) complex numbers and to compute the addition of N complex numbers.

```
# Class to represent a complex number
class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    # Method to display the complex number
    def __str__(self):
        sign = '+' if self.imag >= 0 else '-'
        return f"{self.real} {sign} {abs(self.imag)}i"

# Function to add two complex numbers
def add_complex(c1, c2):
    return Complex(c1.real + c2.real, c1.imag + c2.imag)

# Main program
N = int(input("Enter the number of complex numbers ( $N \geq 2$ ): "))
if N < 2:
    print("N must be at least 2")
else:
    complex_list = []

    # Read N complex numbers
    for i in range(N):
        print(f"Enter complex number {i+1}:")
        real = float(input("  Real part: "))
        imag = float(input("  Imaginary part: "))
        complex_list.append(Complex(real, imag))

    # Initialize sum with first complex number
    total = complex_list[0]

    # Add remaining complex numbers
    for i in range(1, N):
        total = add_complex(total, complex_list[i])

    print("\nSum of all complex numbers =", total)
```

Output:

```
Enter the number of complex numbers ( $N \geq 2$ ): 3
Enter complex number 1:
  Real part: 2
  Imaginary part: 3
```

Enter complex number 2:

Real part: 4

Imaginary part: -1

Enter complex number 3:

Real part: 1

Imaginary part: 2

Sum of all complex numbers = $7 + 4i$

9) Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.

```
import string
```

```
# Function to analyze text
```

```
def analyze_text(text):
```

```
    # Remove punctuation for word frequency
```

```
    translator = str.maketrans("", string.punctuation)
```

```
    cleaned_text = text.translate(translator).lower()
```

```
    # Split into words
```

```
    words = cleaned_text.split()
```

```
    # Word frequency
```

```
    word_freq = {}
```

```
    for word in words:
```

```
        if word in word_freq:
```

```
            word_freq[word] += 1
```

```
        else:
```

```
            word_freq[word] = 1
```

```
    # Longest word
```

```
    longest_word = max(words, key=len) if words else ""
```

```
    # Number of sentences (split by '.', '!', '?')
```

```
    sentences = [s for s in text.replace('!', '.').replace('?', '.').split('.') if s.strip()]
```

```
    num_sentences = len(sentences)
```

```
    # Total number of words
```

```
    total_words = len(words)
```

```
    # Average word length
```

```
    avg_word_length = sum(len(word) for word in words) / total_words if total_words > 0 else
```

```
0
```

```
    # Display results
```

```
    print("\n--- Text Analysis Report ---")
```

```
    print(f"Total sentences      : {num_sentences}")
```

```

print(f"Total words      : {total_words}")
print(f"Longest word      : {longest_word}")
print(f"Average word length : {avg_word_length:.2f}")
print("\nWord Frequencies:")
for word, freq in sorted(word_freq.items(), key=lambda x: x[1], reverse=True):
    print(f"{word} : {freq}")

```

```

# Main program
paragraph = input("Enter a paragraph:\n")
analyze_text(paragraph)

```

Output:

Enter a paragraph:
Python is fun. Python is powerful! Python is easy to learn.

```

--- Text Analysis Report ---
Total sentences      : 3
Total words          : 9
Longest word         : powerful
Average word length  : 4.22

```

Word Frequencies:

```

python : 3
is : 3
fun : 1
powerful : 1
easy : 1
to : 1
learn : 1

```

10) Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.

```

import csv

# Function to read CSV into dictionary
def read_csv_to_dict(filename):
    data = {}
    with open(filename, 'r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            for key, value in row.items():
                if key not in data:
                    data[key] = []
                # Convert numeric values to float if possible
                try:
                    data[key].append(float(value))

```

```

        except ValueError:
            data[key].append(value)
    return data

# Function to summarize a numeric column
def summarize_column(data, column):
    if column not in data:
        print("Column not found!")
        return

    # Filter numeric values
    numeric_values = [v for v in data[column] if isinstance(v, (int, float))]

    if not numeric_values:
        print("No numeric data found in this column.")
        return

    maximum = max(numeric_values)
    minimum = min(numeric_values)
    average = sum(numeric_values) / len(numeric_values)

    print(f"\nSummary for column '{column}':")
    print(f"Maximum = {maximum}")
    print(f"Minimum = {minimum}")
    print(f"Average = {average:.2f}")

# Main program
filename = input("Enter CSV file name (with extension): ")
data = read_csv_to_dict(filename)

print("\nColumns available in the file:")
for col in data.keys():
    print(col)

while True:
    column = input("\nEnter column name to summarize (or 'exit' to quit): ")
    if column.lower() == 'exit':
        print("Exiting program.")
        break
    summarize_column(data, column)

```

Output:

Example CSV (covid.csv):

Code

```
Country,Confirmed,Deaths,Recovered
USA,50000,1000,20000
India,40000,500,15000
Brazil,30000,800,10000
```

Sample Run:

Code

```
Enter CSV file name (with extension): covid.csv

Columns available in the file:
Country
Confirmed
Deaths
Recovered

Enter column name to summarize (or 'exit' to quit): Confirmed

Summary for column 'Confirmed':
Maximum = 50000.0
Minimum = 30000.0
Average = 40000.00
```

11) Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).

Student Grade Tracker

List to store student data
students = []

Read student data
num_students = int(input("Enter the number of students: "))

for i in range(num_students):
 name = input(f"\nEnter name of student {i+1}: ")
 marks = float(input(f"Enter marks of {name}: "))
 students.append({'name': name, 'marks': marks})

Display all student data
print("\n--- Student Records ---")
for student in students:
 print(f"{student['name']} : {student['marks']}")

```

# Calculate average marks
total_marks = sum(student['marks'] for student in students)
average_marks = total_marks / num_students
print(f"\nAverage Marks = {average_marks:.2f}")

# Find topper
topper = max(students, key=lambda x: x['marks'])
print(f"Topper = {topper['name']} with {topper['marks']} marks")

# Optional: Display students sorted by marks
students_sorted = sorted(students, key=lambda x: x['marks'], reverse=True)
print("\n--- Students Sorted by Marks (Highest to Lowest) ---")
for student in students_sorted:
    print(f"{student['name']} : {student['marks']}")

```

Output:

Enter the number of students: 4

Enter name of student 1: Alice
Enter marks of Alice: 85
Enter name of student 2: Bob
Enter marks of Bob: 92
Enter name of student 3: Charlie
Enter marks of Charlie: 78
Enter name of student 4: Diana
Enter marks of Diana: 88

--- Student Records ---
Alice : 85.0
Bob : 92.0
Charlie : 78.0
Diana : 88.0

Average Marks = 85.75
Topper = Bob with 92.0 marks

--- Students Sorted by Marks (Highest to Lowest) ---
Bob : 92.0
Diana : 88.0
Alice : 85.0
Charlie : 78.0

12) Develop a program to display contents of a folder recursively (Directory) having sub-folders and files (name and type).

```

import os

# Function to display folder contents recursively
def display_folder_contents(path, indent=0):

```

```

try:
    # List all items in the directory
    for item in os.listdir(path):
        item_path = os.path.join(path, item)
        if os.path.isdir(item_path):
            print(" " * indent + f"[DIR] {item}")
            # Recursively display sub-folder contents
            display_folder_contents(item_path, indent + 1)
        else:
            print(" " * indent + f"[FILE] {item}")
except PermissionError:
    print(" " * indent + "Permission Denied:", path)

# Main program
directory = input("Enter the directory path to display: ")
print(f"\nContents of '{directory}':\n")
display_folder_contents(directory)

```

Output:

Enter the directory path to display: test_folder

Contents of 'test_folder':

```

[DIR] Projects
[FILE] notes.txt
[DIR] Python
[FILE] app.py
[FILE] data.csv
[FILE] todo.txt
[DIR] Images
[FILE] photo1.jpg
[FILE] photo2.png

```